Information Web Retrieval and Analytics

Lecture 2: NLP Foundations I: Bag-of-Words and Language Models

Institute of Technology of Cambodia · Department of Applied Mathematics and Statistics

Today's agenda

- **Bag-of-words**
  - Idea, 3 worked examples, link to the inverted index, limits
- **N-grams**
  - Extracting n-grams; why they keep local word order
- **N-gram language models**
  - 2 worked examples; perplexity; smoothing

Bag-of-words

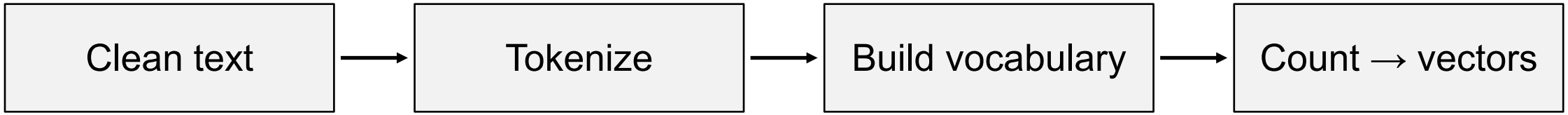
- BoW: is a method to extract features from text documents. These features can be used for training machine learning algorithms. It creates a vocabulary of all the unique words occurring in all the documents in the training set.



Figure 11.2 Intuition of the classic vector space model applied to a single document. The position of the words is ignored (the *bag-of-words* assumption) and we make use of the frequency of each word.

Bag-of-words: the idea

- Represent a document by its **term counts**; ignore word order
  - Only *which* words occur and *how often* matter, not *where* they occur
- Each document → a **count vector** over the vocabulary V
  - One position per vocabulary word: |V| positions, almost all 0 (sparse)
- Basis of TF-IDF, BM25, query likelihood, and text classification
  - The ranking models of the next weeks all start from these counts



Manning, Raghavan & Schütze (2008), section 6.2

BoW Example 1: step by step

S1: *Welcome to Great Learning, Now start learning* S2: *Learning is a good practice*

Step 1 · lower-case, remove punctuation and stop words (*to, is, a*)

```
S1: welcome great learning now start learning
S2: learning good practice
```

Step 2 · vocabulary = every distinct word: [welcome, great, learning, now, start, good, practice]

Step 3 · count each vocabulary word in each sentence

	welcome	great	learning	now	start	good	practice
S1	1	1	2	1	1	0	0
S2	0	0	1	0	0	1	1

Why it works: both vectors have length 7 = |V|, so sentences of different lengths can be compared. *Learning* and *learning* count as one word.

BoW Example 2: three documents

d1: *Mary likes to see sun rise. John likes to play under bright sun.*

d2: *The bright sun rise early.* d3: *See the moon rise*

	mary	likes	to	see	sun	rise	john	play	under	bright	the	early	moon
d1	1	2	2	1	2	1	1	1	1	1	0	0	0
d2	0	0	0	0	1	1	0	0	0	1	1	1	0
d3	0	0	0	1	0	1	0	0	0	0	1	0	1

- Vocabulary from **all** documents: 13 words
  - Every document gets a vector with the same 13 positions, even d3 with only 4 words
- Counts can be > 1: *likes*, *to*, *sun* in d1
  - BoW keeps *how often* a word occurs; *See* and *see* are one word after lower-casing
- Mostly zeros, even here → store only non-zero entries
  - With a real vocabulary of 100,000 words, over 99.9% of each vector is 0

BoW Example 3: from matrix to inverted index

Doc	Text
D1	the cat sat on the mat
D2	the dog sat on the log
D3	the cat chased the dog

	the	cat	sat	on	mat	dog	log	chased
D1	2	1	1	1	1	0	0	0
D2	2	0	1	1	0	1	1	0
D3	2	1	0	0	0	1	0	1

- Read a **column**, drop the zeros → a postings list
  - *cat* → D1 (1), D3 (1) *dog* → D2 (1), D3 (1) *the* → D1 (2), D2 (2), D3 (2)
- Inverted index = sparse storage of the BoW matrix
  - Keep only non-zero cells, grouped by term: a query reads just the columns of its words

Bag-of-words: strengths and limits

Limit	Example	Fix
Word order lost	*dog bites man* = *man bites dog* = [1, 1, 1]	n-grams (next), positional index (week 3)
Raw counts	long documents and frequent words win	TF-IDF, length normalization (week 4)
Synonyms are separate	*car* ≠ *automobile*	dense embeddings (week 10)

- **Strengths**: simple, fast, sparse
  - Easy to compute; stored as an inverted index, so a query is answered in milliseconds
- **Main weakness**: order and meaning are lost
  - Two texts with the same words always get the same vector, whatever they mean

N-gram

- N-gram: is a fundamental concept in NLP, play a pivotal role in capturing patterns and relationships within a sequence of words.

- Examples:
  - Unigrams (1-grams): Single words, e.g., “cat,” “dog.”
  - Bigrams (2-grams): Pairs of consecutive words, e.g., “natural language,” “deep learning.”
  - Trigrams (3-grams): Triplets of consecutive words, e.g., “machine learning model,” “data science approach.”
  - 4-grams, 5-grams, etc.: Sequences of four, five, or more consecutive words.

N-gram Example 1: extracting n-grams

- n-gram: a sequence of **n consecutive** words (or characters)
  - Slide a window of n words over the text, moving one word at a time

n	Name	n-grams of *This is a sentence*	Count
1	unigrams	this · is · a · sentence	4
2	bigrams	this is · is a · a sentence	3
3	trigrams	this is a · is a sentence	2

- A text of L words has **L − n + 1** n-grams
  - *This is a sentence*: L = 4, so 4 − 2 + 1 = 3 bigrams and 2 trigrams
- Character n-grams: *data* → *da · at · ta*
  - Useful for spelling correction and for languages without spaces

N-grams keep local word order

	Unigrams (BoW)	Bigrams
dog bites man	dog · bites · man	dog bites · bites man
man bites dog	dog · bites · man	man bites · bites dog

- Same unigrams, **different bigrams**: order is back
  - A bigram records which word comes right after which
- Phrases are n-grams: *new york*, *machine learning*
  - Matching the bigram, not the two words, finds the phrase (phrase queries, week 3)
- Language models pad sentences with <s> and </s>
  - <s> I am Sam </s> → <s> I · I am · am Sam · Sam </s>: learn how sentences start and end
- Larger n: more context, but **sparser counts**
  - Most long n-grams never appear in the training data

What is a language model?

- Assigns a probability to text: $P(w_1, w_2, \ldots, w_n)$
  - Fluent text gets high probability: P(*I love data*) > P(*data love I*)
- Or predicts the next word: $P(w_n \mid w_1 \ldots w_{n-1})$
  - *information retrieval is … useful* is more likely than *banana*
- Used wherever text is produced or corrected
  - speech recognition, spelling correction, translation, query auto-completion
- LLMs (GPT, …) are very large neural language models
  - Same task, predicting the next word, with neural networks and long contexts

Jurafsky & Martin (2026), ch. 3

Chain rule and the Markov assumption

- **Chain rule** (exact):
  - Probability of a sentence = each word given all the words before it, multiplied

$$P(w_1 \dots w_n) = P(w_1) \cdot P(w_2 \mid w_1) \cdot P(w_3 \mid w_1\ w_2) \cdot \dots \cdot P(w_n \mid w_1 \dots w_{n-1})$$

- Problem: long histories are **never seen** in training data
  - We cannot count *information retrieval is a field of computer* … → science
- **Markov assumption**: only the last word matters (bigram)
  - Replace the long history by the previous word, which we can count

$$P(w_k \mid w_1 \dots w_{k-1}) \approx P(w_k \mid w_{k-1})$$
$$P(w_1 \dots w_n) \approx \prod_k P(w_k \mid w_{k-1})$$

- n-gram model: condition on the last **n − 1** words (bigram: 1, trigram: 2)

N-gram Example 2: bigram probabilities (MLE)

$$P(w_n \mid w_{n-1}) = C(w_{n-1}\ w_n) / C(w_{n-1})$$

Training corpus (3 sentences):

```
<s> I am Sam </s>
<s> Sam I am </s>
<s> I do not like green
    eggs and ham </s>
```

How to read it: *I* occurs 3 times and is followed by *am* twice, so P(am | I) = 2/3. C = count in the corpus.

Bigram	Counts	P
P(I | <s>)	2 / 3	0.67
P(Sam | <s>)	1 / 3	0.33
P(am | I)	2 / 3	0.67
P(do | I)	1 / 3	0.33
P(Sam | am)	1 / 2	0.50
P(</s> | Sam)	1 / 2	0.50

Example corpus from Jurafsky & Martin (2026), section 3.1

N-gram Example 2 (cont.): sentence probability

```
P(<s> I am Sam </s>)
  = P(I|<s>)  · P(am|I)  · P(Sam|am)  · P(</s>|Sam)
  = 2/3 · 2/3 · 1/2 · 1/2
  = 1/9 ≈ 0.111
```

- Multiply one bigram probability per word, left to right
  - Include the start <s> and the end </s>: 4 factors for a 3-word sentence
- In practice: add **log probabilities**
  - $\log(p_1 \cdot p_2 \cdot p_3) = \log p_1 + \log p_2 + \log p_3$
  - Avoids underflow: a product of many small numbers rounds to 0 on a computer

N-gram Example 3: a small search corpus

Training corpus

```
<s> I love search engines </s>
<s> I love data </s>
<s> search engines index data </s>
```

Sentence probability

P(<s> I love data </s>)
= P(I|<s>) · P(love|I) · P(data|love) · P(</s>|data)
= 2/3 · 1 · 1/2 · 1 = **1/3 ≈ 0.33**

Denominators are counts of the first word: <s> = 3, *I* = 2, *love* = 2, *search* = 2, *engines* = 2, *index* = 1, *data* = 2.

Bigram	Counts	P
P(I | <s>)	2 / 3	0.67
P(search | <s>)	1 / 3	0.33
P(love | I)	2 / 2	1
P(search | love)	1 / 2	0.5
P(data | love)	1 / 2	0.5
P(engines | search)	2 / 2	1
P(</s> | engines)	1 / 2	0.5
P(index | engines)	1 / 2	0.5
P(data | index)	1 / 1	1
P(</s> | data)	2 / 2	1

N-gram Example 3 (cont.): what the model learned

Sentence	Product of bigram probabilities	P
I love data	2/3 · 1 · 1/2 · 1	1/3
I love search engines	2/3 · 1 · 1/2 · 1 · 1/2	1/6
I love search engines index data (*never seen*)	2/3 · 1 · 1/2 · 1 · 1/2 · 1 · 1	1/6
data love I (*same words as row 1*)	P(data | <s>) = 0/3	**0**

- **Generalizes**: new sentences get probability from seen bigrams
  - Row 3 is not in the corpus, but each of its bigrams is
- **Order matters**: same bag of words, different probability
  - Rows 1 and 4 have the same words; only the order differs
- **Zeros**: one unseen bigram → probability 0
  - *data* never starts a sentence, so the whole product is 0 (next slide)

Your turn: P(<s> search engines index data </s>) = ?

The zero-probability problem

```
P(<s> I like green eggs </s>)  needs  P(like | I)
P(like | I) = C(I like) / C(I) = 0 / 3 = 0
→ the whole sentence gets probability 0
```

- Most possible n-grams **never occur** in training data
  - A corpus is finite: many correct bigrams like *I like* are simply missing
- One unseen bigram makes the whole product zero
  - and perplexity cannot be computed (division by zero)
- IR: a document missing one query word would score 0
  - even if it matches every other query word very well
- Solution: **smoothing**
  - Move a little probability from seen n-grams to unseen ones

Evaluating language models: perplexity

$$PP(W) = P(w_1\ w_2 \ldots w_N)^{-1/N}$$

- Inverse probability, normalized by length: **lower is better**
  - A good model gives the test text high probability, so low perplexity
- Example: PP = $(1/9)^{-1/4} = 9^{1/4} \approx$ **1.73**
  - N = 4 tokens: *I, am, Sam, </s>* (we do not count <s>)
- Intuition: weighted average **branching factor**
  - At each step the model hesitates between about 1.7 words

WSJ test set	Unigram	Bigram	Trigram
Perplexity	962	170	109

Jurafsky & Martin (2026), section 3.3; models trained on 38 million words of the Wall Street Journal

Laplace (add-one) smoothing

$$P_{\text{Laplace}}(w_n \mid w_{n-1}) = (C(w_{n-1}\, w_n) + 1) / (C(w_{n-1}) + V)$$

Our corpus: V = 11, C(I) = 3	MLE	Laplace
P(like | I)	0 / 3 = 0	(0 + 1) / (3 + 11) = 1/14 ≈ 0.071
P(am | I)	2 / 3 ≈ 0.667	(2 + 1) / (3 + 11) = 3/14 ≈ 0.214

- Why + V? We added 1 for each of the V possible next words
  - so the probabilities after a word still sum to 1
- No more zeros, but **too much mass** moves to unseen events
  - P(am | I) drops from 0.67 to 0.21, although *I am* is common
- **Add-k**: add a fraction k < 1 instead of 1
  - e.g. k = 0.5 or 0.01 moves less mass; choose k on held-out data

Summary

- **Bag-of-words**: a document = a count vector; order is ignored
  - the document-term matrix, stored by columns, is the inverted index
- **N-grams**: sequences of n words; they keep local word order
- **N-gram LMs**: P(text) from counts (MLE) with the Markov assumption
  - evaluate with **perplexity**; fix zeros with **smoothing**
- **TP**: preprocessing pipeline + bigram model with add-k smoothing
- **Next week**: Boolean retrieval and the inverted index

References

- Jurafsky, D. & Martin, J. H. (2026). *Speech and Language Processing* (3rd ed. draft). Ch. 3 N-gram Language Models (ch. 2 Words and Tokens for the TP).
- Manning, C. D., Raghavan, P. & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. Section 6.2; ch. 12 (ch. 2 for the TP).



Thank you!